

FinShield Insurance Operations Cloud

Complete AWS Cloud Engineering Project Documentation

Field	Details
Project domain	Insurance Technology & Risk Management
Primary objective	Demonstrate AWS cloud infrastructure, Linux administration, networking, monitoring, security, automation and cost-awareness.
Application type	Small web application deployed on AWS EC2 using Docker Compose
Current access link	http://13.232.210.150
AWS region	ap-south-1 (Asia Pacific - Mumbai)
Instance	Ubuntu EC2 instance, free-tier style t2.micro/t3.micro equivalent depending on account/region eligibility
Prepared for	Academic submission and viva explanation
Prepared date	2026-06-20

Important: The current public IP based link can change after stopping and starting the EC2 instance unless an Elastic IP is attached. Before viva, start the instance, fetch the new public IP, and update the demo link.

Table of Contents

1. Executive Summary
2. Problem Statement and Expected Outcome
3. Scope of the Project
4. Requirements Mapping
5. End-to-End Architecture
6. AWS Services and Concepts Used
7. Implementation Method Overview
8. AWS Console UI Deployment Steps
9. AWS CLI Command Documentation
10. SSH Access and Key Pair Usage
11. Linux Administration Steps
12. Code Transfer Using Git and SCP

13. Docker and Docker Compose Deployment
14. Database: MySQL/MariaDB Concept and Backup
15. Deployment Verification and CRUD API Testing
16. Monitoring and Automation Scripts
17. Security Hardening
18. Pricing and Cost Control
19. Stop, Resume and Cleanup Workflow
20. Troubleshooting Guide
21. Viva Concept Guide
22. Conclusion
23. References

1. Executive Summary

FinShield Insurance Operations Cloud is a domain-focused cloud engineering project. The application itself is intentionally small, but the deployment demonstrates the core AWS cloud concepts expected in the case study: architecture design, virtual networking, EC2/VM deployment, Linux administration, Docker containerization, database handling, monitoring, automation, security and pricing awareness.

The system was deployed on an AWS EC2 Ubuntu server inside a custom VPC. The EC2 instance was placed in a public subnet, protected by a Security Group, accessed using SSH key-based authentication, configured with Docker and Docker Compose, and used to run the FinShield application stack. The application exposes a public web interface and authenticated APIs for insurance-related operational records.

Note: In viva, the key message is: the project is not judged only by app complexity; it is judged by how clearly it demonstrates cloud computing implementation.

2. Problem Statement and Expected Outcome

Business domain: Insurance Technology & Risk Management.

The organization needs a centralized cloud platform to replace disconnected spreadsheets, manual workflows and isolated reporting environments. The proposed system should support operational records, reporting, monitoring, secure access and future expansion.

Expected Area	How This Project Demonstrates It
---------------	----------------------------------

Cloud Architecture	A VPC-based AWS architecture with public subnet, route table, internet gateway, security group and EC2 server.
Linux Administration	Ubuntu package updates, users/groups, file permissions, logs, cron and troubleshooting commands.
EC2/VM Deployment	An EC2 instance runs the deployed application stack.
MySQL/MariaDB	The application database runs in a container and is backed up using scripts.
Docker	Application components are containerized and started using Docker Compose.
VPC & Networking	Custom CIDR, subnet, internet route and firewall rules demonstrate AWS networking.
Monitoring	Linux and Docker monitoring commands plus custom scripts.
Automation Scripts	Cron jobs for backup, monitoring and maintenance.
Pricing Analysis	Free-tier, EC2 stop/start, EBS storage and public IPv4 considerations.
Reporting Dashboards	App UI includes dashboards/reports for insurance operations.

3. Scope of the Project

This is a cloud engineering demonstration, not a full-scale production insurance platform. The scope is deliberately practical and academic.

In Scope	Out of Scope
Deploy a working web app on AWS	Building a bank-grade/enterprise-grade production insurance suite
Use EC2, VPC, Security Groups and SSH	Full multi-region active-active disaster recovery
Run app using Docker Compose	Kubernetes/EKS production orchestration
Use MySQL/MariaDB style database storage	Managed production RDS high availability
Demonstrate monitoring, logs and cron	Full observability platform with CloudWatch dashboards, alerts and SIEM
Provide pricing estimate and cost controls	Enterprise financial commitment planning

4. Requirements Mapping

Requirement	Implemented Component	Viva Explanation
Scalability	Dockerized services on EC2; future path to ALB/Auto Scaling/ECS	The current instance is small, but containerization makes future scaling easier.
High Availability	Discussed as future improvement using multiple AZs and Load Balancer	The demo is single-instance; production would need multi-AZ.
Secure Access	IAM user, SSH key pair, Security Group, optional SSH IP restriction	Access is controlled at both AWS identity and network layers.
Centralized Visibility	Dashboard, API health checks, logs and monitoring commands	Admins can view service health and operational data.
Operational Efficiency	Docker Compose and cron scripts	Deployment and maintenance are repeatable.
Backup & Recovery	Database backup script and backup directory	Backup enables restoration if data is lost.
Cost Optimization	Stop/start instance, free-tier instance, avoid NAT/ALB unless needed	Costs are minimized for academic/demo use.

5. End-to-End Architecture

The deployed architecture follows a simple but complete public EC2 pattern. It is appropriate for a learning project and viva demonstration.

Architecture Flow

```
User Browser / curl
    |
    | HTTP request: http://<EC2-Public-IP>
    v
Public Internet
    |
    v
Internet Gateway
    |
    v
FinShield VPC: 10.0.0.0/16
    |
    v
Public Subnet: 10.0.1.0/24
    |
    v
Security Group: allows 22, 80, 443
    |
    v
EC2 Ubuntu Server
    |
    v
Docker Engine + Docker Compose
    |
    +--> Nginx / Frontend container
    +--> Backend API container
    +--> MySQL/MariaDB database container
```

5.1 Request Flow

25. A user enters the EC2 public IP in the browser.
26. The request travels through the internet and reaches the AWS Internet Gateway attached to the VPC.
27. The route table allows internet traffic to reach the public subnet.
28. The Security Group checks whether the requested port is allowed.
29. For web traffic, port 80 or 443 is allowed; for administration, port 22 is used for SSH.
30. The EC2 server receives traffic and Docker forwards it to the correct container.
31. Nginx serves frontend assets or forwards API requests to the backend.
32. The backend communicates with the database container over Docker's internal network.

6. AWS Services and Concepts Used

Term	Meaning	Used For
IAM	Identity and Access Management	Create a deployer user with EC2/VPC permissions.
VPC	Virtual Private Cloud	Private network boundary for AWS resources.
CIDR	IP range notation	10.0.0.0/16 for VPC and 10.0.1.0/24 for subnet.
Subnet	Smaller network inside a VPC	Place EC2 in a public subnet.
Internet Gateway	VPC component that enables internet access	Allows public web access to the instance.
Route Table	Rules that decide where traffic goes	Route 0.0.0.0/0 to the Internet Gateway.
Security Group	Virtual firewall	Allow SSH, HTTP and HTTPS.
EC2	Elastic Compute Cloud virtual machine	Runs Ubuntu, Docker and the application.
AMI	Amazon Machine Image	Base OS image used to launch EC2.
Key Pair	Public/private SSH authentication pair	Allows secure login using .pem file.
EBS	Elastic Block Store	Root disk storing OS, app files, Docker images and data.

7. Implementation Method Overview

The project can be implemented using either AWS Console UI or AWS CLI. Both approaches create the same infrastructure. The Console is easier to understand visually, while the CLI is better for repeatability and documentation.

Method	Best For	Limitation
AWS Console UI	Learning, screenshots, viva explanation	Manual and less repeatable
AWS CLI	Fast setup, repeatable commands, documentation	Requires command accuracy
Terraform/CloudFormation	Production infrastructure as code	Extra learning curve; not necessary for this project

8. AWS Console UI Deployment Steps

These are the high-level UI steps to reproduce the project without AWS CLI for infrastructure creation.

8.1 Create IAM User

33. Open AWS Console and search for IAM.
34. Go to Users -> Create user.
35. Username: finshield-deployer.
36. Attach policies directly: AmazonEC2FullAccess and AmazonVPCFullAccess.
37. Create the user.
38. Open the user -> Security credentials -> Create access key -> Command Line Interface use case.
39. Download the CSV and store it safely. Do not upload it to GitHub.

8.2 Create VPC and Networking

40. Open VPC service in AWS Console.
41. Create VPC with name FinShield-VPC and CIDR 10.0.0.0/16.
42. Create subnet FinShield-Public with CIDR 10.0.1.0/24 in ap-south-1a.
43. Create and attach an Internet Gateway named FinShield-IGW to the VPC.
44. Open Route Tables, select the VPC route table, and add route 0.0.0.0/0 targeting the Internet Gateway.
45. Associate the route table with FinShield-Public subnet.
46. Enable auto-assign public IPv4 for the public subnet.

8.3 Create Security Group

47. Open EC2 -> Security Groups -> Create security group.
48. Name: FinShield-SG. VPC: FinShield-VPC.
49. Inbound rules: SSH TCP 22, HTTP TCP 80, HTTPS TCP 443.
50. For SSH, prefer My IP instead of 0.0.0.0/0 when possible.
51. Outbound rules can remain default for this project.

8.4 Launch EC2 Instance

52. Open EC2 -> Instances -> Launch instance.
53. Name: FinShield-Server.
54. Choose Ubuntu Server 22.04 or compatible Ubuntu LTS AMI.
55. Choose free-tier eligible instance type such as t2.micro/t3.micro depending on account/region availability.
56. Create or select key pair FinShield-Key and download the .pem file.

- 57. Network: FinShield-VPC. Subnet: FinShield-Public. Auto-assign public IP: Enable.
- 58. Security Group: FinShield-SG.
- 59. Storage: default 8-20 GB gp2/gp3 volume is enough for the demo.
- 60. Launch instance and copy the public IPv4 address.

9. AWS CLI Command Documentation

The following commands document the CLI implementation used for the project. Run AWS CLI commands from the local Mac terminal, not from inside EC2, unless specified.

9.1 Configure AWS CLI

```
brew install awscli
aws configure
# Access Key ID: from IAM CSV
# Secret Access Key: from IAM CSV
# Default region: ap-south-1
# Default output: json

aws sts get-caller-identity
aws configure get region
```

9.2 Create VPC and Networking

```

VPC_ID=$(aws ec2 create-vpc --cidr-block 10.0.0.0/16 --tag-specifications
'ResourceType=vpc,Tags=[{Key=Name,Value=FinShield-VPC}]' --query 'Vpc.VpcId' --output
text)

echo $VPC_ID

SUBNET_ID=$(aws ec2 create-subnet --vpc-id $VPC_ID --cidr-block 10.0.1.0/24
--availability-zone ap-south-1a --tag-specifications
'ResourceType=subnet,Tags=[{Key=Name,Value=FinShield-Public}]' --query 'Subnet.SubnetId'
--output text)

echo $SUBNET_ID

IGW_ID=$(aws ec2 create-internet-gateway --tag-specifications
'ResourceType=internet-gateway,Tags=[{Key=Name,Value=FinShield-IGW}]' --query
'InternetGateway.InternetGatewayId' --output text)

echo $IGW_ID

aws ec2 attach-internet-gateway --internet-gateway-id $IGW_ID --vpc-id $VPC_ID

ROUTE_TABLE_ID=$(aws ec2 describe-route-tables --filters "Name=vpc-id,Values=$VPC_ID"
--query 'RouteTables[0].RouteTableId' --output text)

echo $ROUTE_TABLE_ID

aws ec2 create-route --route-table-id $ROUTE_TABLE_ID --destination-cidr-block
0.0.0.0/0 --gateway-id $IGW_ID

aws ec2 associate-route-table --route-table-id $ROUTE_TABLE_ID --subnet-id $SUBNET_ID

aws ec2 modify-subnet-attribute --subnet-id $SUBNET_ID --map-public-ip-on-launch

```

9.3 Create Security Group

```

SG_ID=$(aws ec2 create-security-group --group-name FinShield-SG --description
"FinShield firewall rules" --vpc-id $VPC_ID --query 'GroupId' --output text)

echo $SG_ID

aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 22
--cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 80
--cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 443
--cidr 0.0.0.0/0

```

9.4 Create Key Pair and Launch Instance

```
aws ec2 create-key-pair --key-name FinShield-Key --query 'KeyMaterial' --output text
> FinShield-Key.pem

chmod 400 FinShield-Key.pem

AMI_ID=$(aws ec2 describe-images --owners 099720109477 --filters
"Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"
"Name=virtualization-type,Values=hvm" "Name=architecture,Values=x86_64"
"Name=root-device-type,Values=ebs" --query 'Images | sort_by(@,
&CreationDate)[-1].ImageId' --output text)

echo $AMI_ID

INSTANCE_ID=$(aws ec2 run-instances --image-id $AMI_ID --instance-type t2.micro
--key-name FinShield-Key --security-group-ids $SG_ID --subnet-id $SUBNET_ID
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=FinShield-Server}]'
--query 'Instances[0].InstanceId' --output text)

echo $INSTANCE_ID

aws ec2 wait instance-running --instance-ids $INSTANCE_ID

PUBLIC_IP=$(aws ec2 describe-instances --instance-ids $INSTANCE_ID --query
'Reservations[0].Instances[0].PublicIpAddress' --output text)

echo $PUBLIC_IP
```

10. SSH Access and Key Pair Usage

SSH is used to securely connect from the local machine to the EC2 Ubuntu server. The .pem file is the private key. AWS keeps the public key in the EC2 instance metadata at launch time.

```
ssh -i FinShield-Key.pem ubuntu@$PUBLIC_IP

# Example using current public IP
ssh -i FinShield-Key.pem ubuntu@13.232.210.150
```

Note: If the terminal asks 'Are you sure you want to continue connecting?', type yes. This happens the first time your Mac trusts the new server fingerprint.

11. Linux Administration Steps

These commands run inside the EC2 Ubuntu server after SSH login.

```

sudo apt update && sudo apt upgrade -y

sudo groupadd finshield
sudo useradd -m -s /bin/bash -g finshield finshield-app
sudo useradd -m -s /bin/bash -G finshield,sudo finshield-admin

sudo mkdir -p /opt/finshield /var/log/finshield
sudo chown -R finshield-app:finshield /opt/finshield
sudo chmod -R 750 /opt/finshield

```

Command/Concept	Purpose
apt update	Refresh package list.
apt upgrade	Apply latest security/package updates.
groupadd	Create project group for shared access.
useradd	Create separate app/admin Linux users.
/opt/finshield	Application deployment directory.
/var/log/finshield	Log directory for backup/monitoring output.
chown	Change owner/group.
chmod 750	Owner full access, group read/execute, others no access.

12. Code Transfer Using Git and SCP

12.1 Git Clone Method

```

cd /opt/finshield
sudo -u finshield-app git clone <your-github-repo-url> .

# If the repo creates nested folder FinShield:
cd /opt/finshield/FinShield
ls -la

```

12.2 SCP Method

SCP is used when the code is present only on the local Mac and not on GitHub. The SCP command runs from the Mac terminal.

```
# From Mac terminal

scp -i FinShield-Key.pem -r "/Users/shivammishra/Desktop/AWS/"*
ubuntu@$PUBLIC_IP:/tmp/finshield/

# Then inside EC2

sudo cp -r /tmp/finshield/* /opt/finshield/
sudo chown -R finshield-app:finshield /opt/finshield/
sudo chmod -R 750 /opt/finshield
```

13. Docker and Docker Compose Deployment

Docker packages the application into containers. Docker Compose starts multiple containers using one YAML file, such as frontend/nginx, backend API and database.

13.1 Install Docker on Ubuntu

```
sudo apt install -y apt-transport-https ca-certificates curl software-properties-common

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update

sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin

sudo usermod -aG docker ubuntu
sudo usermod -aG docker finshield-app

exit
```

Note: After `usermod -aG docker`, log out and SSH back in so the group membership is refreshed.

13.2 Start the Application

```
ssh -i FinShield-Key.pem ubuntu@$PUBLIC_IP
cd /opt/finshield/FinShield

docker compose up --build -d

docker ps
docker compose ps
```

Docker Command	Purpose
docker compose up	Create and start all services defined in docker-compose.yml.
--build	Build images before starting containers.
-d	Detached mode; containers run in the background.
docker ps	Show running containers.
docker logs	Inspect logs of a container.

14. Database: MySQL/MariaDB Concept and Backup

The database stores operational records such as policies, claims, users, audit logs and report data. In this project, database service is run as a Docker container for simplicity. In production, a managed database such as Amazon RDS would be preferred for automated backups, patching and high availability.

```
# Check database container logs
docker logs finshield-db --tail 50

# Manual backup test
sudo mkdir -p /opt/finshield/backups
sudo chown -R ubuntu:ubuntu /opt/finshield/backups
bash /opt/finshield/FinShield/backend/scripts/backup.sh
ls -la /opt/finshield/backups
```

15. Deployment Verification and CRUD API Testing

Verification proves that the public network, EC2 server, Docker containers, backend and database are working together.

```
# From Mac terminal
BASE_URL="http://13.232.210.150"

curl $BASE_URL/api/health

TOKEN=$(curl -s -X POST "$BASE_URL/api/auth/login" -H "Content-Type: application/json"
-d '{"username":"admin","password":"admin123"}' | python3 -c "import sys,json;
print(json.load(sys.stdin)['token'])")

echo $TOKEN

curl -s -H "Authorization: Bearer $TOKEN" "$BASE_URL/api/policies" | python3 -m json.tool
```

15.1 Example CRUD Commands

```
# CREATE policy
curl -s -X POST "$BASE_URL/api/policies" -H "Content-Type: application/json" -H
"Authorization: Bearer $TOKEN" -d '{
    "policy_number": "POL-TEST-001",
    "customer_name": "Test Customer",
    "policy_type": "Health Insurance",
    "premium_amount": 25000,
    "coverage_amount": 500000,
    "region": "West",
    "status": "Active"
}' | python3 -m json.tool

# READ policies
curl -s -H "Authorization: Bearer $TOKEN" "$BASE_URL/api/policies" | python3 -m json.tool

# UPDATE policy
curl -s -X PUT "$BASE_URL/api/policies/1" -H "Content-Type: application/json" -H
"Authorization: Bearer $TOKEN" -d '{"premium_amount":30000,"status":"Active"}' | python3
-m json.tool

# DELETE policy
curl -s -X DELETE "$BASE_URL/api/policies/1" -H "Authorization: Bearer $TOKEN" |
python3 -m json.tool
```

16. Monitoring and Automation Scripts

```
# Manual monitoring commands

free -h
df -h
htop
docker stats --no-stream
docker ps

# Make scripts executable
chmod +x /opt/finshield/FinShield/backend/scripts/*.sh

# Log directory
sudo mkdir -p /var/log/finshield
sudo chown -R ubuntu:ubuntu /var/log/finshield
```

16.1 Cron Automation

```
crontab -e

# Add these lines
0 2 * * * /opt/finshield/FinShield/backend/scripts/backup.sh >>
/var/log/finshield/backup.log 2>&1
*/5 * * * * /opt/finshield/FinShield/backend/scripts/monitor.sh >>
/var/log/finshield/monitor.log 2>&1
0 3 * * 0 /opt/finshield/FinShield/backend/scripts/maintenance.sh >>
/var/log/finshield/maintenance.log 2>&1

# Verify
crontab -l

tail -n 50 /var/log/finshield/backup.log
tail -n 50 /var/log/finshield/monitor.log
```

Cron Expression	Meaning	Purpose
0 2 * * *	Every day at 2:00 AM	Database backup
*/5 * * * *	Every 5 minutes	Monitoring script
0 3 * * 0	Every Sunday at 3:00 AM	Weekly maintenance

17. Security Hardening

Security Layer	Implementation
AWS account access	IAM user instead of root for deployment commands.

SSH authentication	.pem key pair instead of password-based login.
Network firewall	Security Group allows only required ports.
Linux permissions	Separate app user, restricted /opt/finshield permissions.
Container isolation	App services run inside Docker containers.
API security	JWT token required for protected routes.
Database exposure	Database should remain internal to Docker network; not exposed publicly.

```
# Disable password SSH login
sudo nano /etc/ssh/sshd_config

# Set:
PasswordAuthentication no

sudo systemctl restart sshd

# Restrict SSH to current IP if desired
MY_IP=$(curl -s https://checkip.amazonaws.com)

aws ec2 revoke-security-group-ingress --group-id $SG_ID --protocol tcp --port 22 --cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress --group-id $SG_ID --protocol tcp --port 22 --cidr ${MY_IP}/32
```

18. Pricing and Cost Control

The goal is to keep the demo low-cost while preserving the ability to resume before viva. Stopping EC2 stops compute charges, but EBS storage can continue to cost because the disk remains stored.

Resource	Cost Behavior	Control Method
EC2 compute	Charged while running; not charged for usage while stopped	Stop when not practicing
EBS volume	Can still charge while EC2 is stopped	Keep until viva; delete after final cleanup
Public IPv4	Public IPv4 has hourly pricing; free-tier rules depend on account/credits	Stop instance when not in use; avoid Elastic IP unless needed
Docker images	Stored as files on EBS, not separate AWS resources	Do not delete before viva unless disk is full

NAT Gateway/Load Balancer	Can be costly if created	Not used in this simple architecture
---------------------------	--------------------------	--------------------------------------

19. Stop, Resume and Cleanup Workflow

19.1 Stop When Not Practicing

```
aws ec2 stop-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-stopped --instance-ids $INSTANCE_ID

aws ec2 describe-instances --instance-ids $INSTANCE_ID --query
'Reservations[0].Instances[0].State.Name' --output text
```

19.2 Resume Before Viva

```
aws ec2 start-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-running --instance-ids $INSTANCE_ID

PUBLIC_IP=$(aws ec2 describe-instances --instance-ids $INSTANCE_ID --query
'Reservations[0].Instances[0].PublicIpAddress' --output text)

echo $PUBLIC_IP
ssh -i FinShield-Key.pem ubuntu@$PUBLIC_IP

cd /opt/finshield/FinShield
docker compose up -d
docker ps
```

19.3 Cleanup After Viva

```
# Only after final submission/demo is done
aws ec2 terminate-instances --instance-ids $INSTANCE_ID
aws ec2 wait instance-terminated --instance-ids $INSTANCE_ID

# Check leftover unattached EBS volumes
aws ec2 describe-volumes --filters "Name=status,Values=available" --query
'Volumes[*].{VolumeId:VolumeId,Size:Size,State:State}' --output table

# Delete an unused volume only if you are sure it is not needed
aws ec2 delete-volume --volume-id vol-your-volume-id

# Check Elastic IPs
aws ec2 describe-addresses --query
'Addresses[*].{PublicIp:PublicIp,AllocationId:AllocationId,InstanceId:InstanceId}'
--output table
```

20. Troubleshooting Guide

Problem	Likely Cause	Fix
aws: command not found	AWS CLI not installed or PATH issue	Install AWS CLI or fix terminal PATH.
Region format error	Entered 'Mumbai AWS region' instead of ap-south-1	Run aws configure set region ap-south-1.
Permission denied publickey	Wrong .pem, wrong user, or bad permissions	Use ubuntu user and chmod 400 key.
SSH timeout	Port 22 blocked, missing public IP, route issue	Check Security Group, subnet public IP and route table.
Browser not loading	Port 80 blocked or container not running	Check Security Group and docker ps.
Backup permission denied	Backup folder missing or not writable	sudo mkdir -p /opt/finshield/backups and chown ubuntu.
backup.log missing	Script ran without redirection or cron has not run	Run script with >> /var/log/finshield/backup.log 2>&1.

21. Viva Concept Guide

21.1 One-Minute Project Explanation

I built and deployed a FinShield Insurance Operations Cloud application on AWS. The application is a small insurance operations portal, but the main objective is cloud infrastructure demonstration. I created a custom VPC, public subnet, internet gateway, route table and security group, launched an Ubuntu EC2 instance, connected using SSH key pair, configured Linux users and permissions, installed Docker, deployed the application with Docker Compose, verified APIs using curl, configured backup and monitoring scripts using cron, and documented pricing/cost control by stopping the instance when not in use.

21.2 High-Probability Viva Questions

Question	Strong Answer
Why did you use AWS?	AWS provides on-demand compute, networking and storage resources. It allowed me to demonstrate real cloud deployment instead of only local execution.
What is VPC?	VPC is a private network inside AWS where my cloud resources live. It gives isolation and control over IP ranges, subnets and routing.
What makes a subnet public?	A subnet is public when its route table has a route to an Internet Gateway and instances have public IPs.
Why use Security Group?	It is a virtual firewall. I allowed only required ports: 22 for SSH, 80 for HTTP and 443 for HTTPS.
Why use Docker?	Docker packages the app with its dependencies and makes deployment repeatable. Docker Compose starts multiple services together.
Why not build a huge app?	The AWS case study focuses on cloud engineering concepts. The app is only a domain-specific workload used to demonstrate deployment, networking, monitoring and security.
What happens when you stop EC2?	Compute stops and containers stop, but EBS disk remains. Data and Docker images are preserved; public IP may change after restart.
What would you improve for production?	Use RDS, HTTPS domain, Load Balancer, Auto Scaling, private subnets, CloudWatch alarms, backups and IAM least privilege.

21.3 Terminology Crash Sheet

Term	Simple Meaning
Cloud	Renting computing resources over the internet.
Region	AWS geographic area, e.g., Mumbai ap-south-1.
Availability Zone	A separate data center area inside a region.
Compute	CPU/RAM server capacity, here EC2.
Storage	Disk/data storage, here EBS and database volume.
Networking	How traffic moves between internet, VPC, subnet and server.
Firewall	Rules that allow or block traffic.
Container	Isolated package running one app/service.
Image	Blueprint used to create a container.
Volume	Persistent storage for containers/databases.
Cron	Linux scheduler for repeated tasks.

22. Conclusion

The FinShield Insurance Operations Cloud project successfully demonstrates core cloud engineering concepts using AWS. The system uses a custom VPC, public subnet, internet gateway, route table, security group, EC2, SSH key-based administration, Linux setup, Docker Compose deployment, database backup, monitoring commands, automation scripts and pricing/cost-control workflow. The project is suitable for academic assessment because it maps directly to the expected AWS case study requirements while keeping the application scope realistic and understandable.

23. References

- AWS VPC Internet Gateway documentation:
https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Internet_Gateway.html
- AWS VPC Route Tables documentation:
https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html
- AWS EC2 stop/start documentation:
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/Stop_Start.html
- AWS EC2 stop/start cost behavior:
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/how-ec2-instance-stop-start-works.html>

- AWS Public IPv4 pricing announcement:
<https://aws.amazon.com/blogs/aws/new-aws-public-ipv4-address-charge-public-ip-insights/>
- AWS EC2 T2 Free Tier instance information:
<https://aws.amazon.com/ec2/instance-types/t2/>
- Amazon EBS pricing: <https://aws.amazon.com/ebs/pricing/>
- Docker Engine on Ubuntu installation documentation:
<https://docs.docker.com/engine/install/ubuntu/>